

Phase 0 讲义：大模型完整生命周期导引

核心目标：这是一份系统化的导引课程，帮助你理解大语言模型从预训练到推理的完整技术栈。包含宏观讲解和微观深化，从基础概念到实现原理，为 Phase 1-4 的深入学习打好基础。

学习时间：5 小时

关键输出：大模型训练到推理的全局视野

📖 序章：为什么要理解大模型的全景？

你即将做的是**指令微调**，但微调不是孤立存在的。它是一个大型系统工程的一部分：

预训练阶段（工业界完成）

↓ [生成基础模型]

微调阶段（你要做的）

↓ [适配下游任务]

推理优化与部署（最终用户使用）

理解全景的好处：

- ☒ 知道你的微调在哪一环
- ☒ 理解微调的限制和机会
- ☒ 做正确的优化决策（为什么用 LoRA 不用 Full FT）
- ☒ 理解评估指标应该怎么选
- ☒ 为面试准备充分的技术深度

🎯 第一阶段：预训练（Pretraining）

核心问题：大模型怎么从零训练？

预训练的目标很简单：用海量文本数据，教模型预测下一个单词。

输入： "The quick brown fox jumps over the ____" 目标： 模型预测 "lazy" 方式： 自回归
因果语言建模 (Causal LM) 损失： 交叉熵损失

这看起来很简单，但背后有巨大的工程和算法挑战。

核心技术栈

技术	作用	为什么重要
Transformer架构	堆叠自注意力层	高效并行、长序列理解
RoPE位置编码	编码位置信息	支持长序列外推、旋转不变性
Flash Attention	高效注意力计算	减少显存占用、加速训练
混合精度 (BF16)	FP32 + FP16 混合	节省显存、加速、保持精度
分布式训练	TP/PP/DP 并行	支持 100B+ 参数模型训练

关键参数：Scaling Law

核心参数关系：

- 模型规模 (parameters) N
- 数据量 (tokens) D
- 计算量 (FLOPs) $C \approx 6ND$

Chinchilla Scaling Law:

- 最优配置: $D \approx 20N$ (数据量约为参数量的20倍)

例子：

- 7B 参数模型 → 最优用 140B tokens 训练
- 70B 参数模型 → 最优用 1400B tokens 训练

直觉：数据不能太少（会欠训），也不能只堆参数（会过度训练）。最优平衡是数据量约为参数量的 20 倍。

预训练的结果

✅ 基础模型 (Base Model)

- 强大的语言理解能力
 - 能预测下一个 token
 - 但不会遵循指令、没有安全对齐
 - 例子: Qwen-VL-7B-base
-

🔄 第二阶段：微调与对齐 (Alignment)

核心问题：预训练完的模型能直接用吗？

不行。因为：

1. ❌ 不遵循指令（你问什么它不一定回答什么）
2. ❌ 没有安全对齐（可能输出有害内容）
3. ❌ 格式不规范（输出格式可能不符合预期）

解决方案：微调 + 对齐

◆ 微调方案 A：SFT（有监督微调）

基本原理

基本原理：

- 数据：高质量的 {指令, 输出} 对
- 目标：让模型学会遵循指令
- 方式：和预训练一样的因果语言建模
- 公式： $\min_{\theta} \sum_i -\log P_{\theta}(y_i|x_i)$

例子（你的任务）：

- 输入 (x): `<video>` + "用一句话描述视频中的摄像机运动"

- 输出 (y): "从左到右的平移运动"
- **SFT**: 让模型学会在给定指令时输出正确答案

SFT 的效果

-  显著改善指令遵循能力
-  改善输出格式
-  但可能过拟合（只学会了 SFT 数据的风格）
-  可能学到错误的偏好

你为什么选 SFT

-  CameraBench 有高质量的 QA 对
-  时间成本低，快速验证
-  对于 VQA 任务足够有效

◆ 微调方案 B: RLHF（强化学习人类反馈）

为什么 SFT 不够

SFT 只能学会"有标注的答案", 但无法学会:

- "什么样的回答更好"
- "在不确定时该怎么办"
- "涉及安全问题时该拒绝"

问题的本质: SFT 数据中通常每个问题只有一个"标准答案", 但在现实中:

- 多个答案可能都是合理的
- 某些答案比另一些"更好" (更有用、更安全、更诚实)
- 模型需要学到这种**偏好**, 而不仅仅是"正确答案"

深层理解：RLHF 的优化目标和实现

RLHF 的优化目标

核心目标：最大化模型输出符合人类偏好的概率

数学表述：

数学表述：

$$\text{maximize } \mathbb{E}_x [\log P_\theta(y_{\text{preferred}}|x) - \log P_\theta(y_{\text{dispreferred}}|x)]$$

直觉：

- 提高优选答案的概率
- 降低非优选答案的概率
- 差异越大越好

为什么需要强化学习

直接优化这个目标很困难，有两个根本问题：

问题 1：梯度何在？

- $y_{\text{preferred}}$ 和 $y_{\text{dispreferred}}$ 都是离散序列
- 无法直接求导

问题 2：太多可能的答案

- 对于一条指令，有无穷多个可能的答案
- 无法把所有答案都标注

强化学习的解决方案：

- 用 Reward Model 代理人类判断（可求导）
- 用采样而非枚举所有答案
- 用策略梯度（Policy Gradient）优化

RLHF 的三阶段详解

Step 1: SFT（有监督微调）

目标：先让模型学会"基本格式"和"遵循指令" 数据：高质量的 {指令, 答案} 对 方法：标准的因果语言建模 结果：模型能输出看起来合理的答案

Step 2: 训练 Reward Model

思路：用一个较小的模型 R_ψ 学习"什么是好答案"

数据格式：

{指令, 答案_A, 答案_B, 标签} 标签：哪个答案更好（由人工标注）

训练方式：Reward Model 是一个二分类器

- 输入：(指令, 答案)
- 输出：一个标量分数（越高越好）

损失函数：

$$L = -\log \sigma(r(x, y_{\text{win}}) - r(x, y_{\text{loss}}))$$

其中 σ 是 sigmoid, r 是 Reward Model 的输出

直觉：

- 如果 $r(\text{优选}) > r(\text{非优选})$, 损失小
- 否则损失大
- 学到的 Reward Model 能判断"什么答案更好"

为什么可行：

- Reward Model 体积小（比 LLM 小）
- 学习一个标量函数比微调整个 LLM 简单
- 偏好判断的模式相对一致

Step 3: 用 RL 优化 LLM

现在我们有：

- 微调后的 LLM (政策 π)
- Reward Model (奖励信号 r)

目标函数：

目标函数：

$$\text{maximize } \mathbb{E}[r(x, y)] - \lambda \cdot \text{KL}(\pi_{\text{new}} || \pi_{\text{SFT}})$$

两项的意义：

- 第一项：最大化 Reward Model 的得分
- 第二项：防止新 LLM 偏离原始 LLM 太远（正则化） 原因：如果只最大化 r ，模型可能找到"作弊方法"高分但输出差

算法：PPO (Proximal Policy Optimization) —— 一种策略梯度方法

大致流程：

For each iteration:

1. 用当前 LLM 采样一批输出 $\{y_1, y_2, \dots\}$
2. 用 Reward Model 评分 $\{r_1, r_2, \dots\}$
3. 计算优势函数 (Advantage) :

$$A_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

4. 用 PPO 更新 LLM:

$$L = -A_t \cdot \log \pi(y_t | x)$$

5. 重复

结果：LLM 学到了"什么答案会被 Reward Model 高分评价" = 学到了人类的偏好！

RLHF 的代价

-  学到人类偏好，输出质量更高
-  需要大量人工标注数据（偏好对）
-  训练复杂（需要 Reward Model + RL）
-  可能过度优化（reward hacking）
-  时间成本高（三个阶段串联）

为什么后来出现 DPO

在 RLHF 之后，研究者发现一个关键洞察：

- 直接用偏好数据优化 LLM，不需要显式的 Reward Model
 - 结果更简单、更稳定、更快
-

◆ 微调方案 C：DPO（直接偏好优化）

核心思想

DPO 的创意：**不需要 Reward Model**，直接用偏好数据优化 LLM。

数据格式：{指令, 优选回答, 非优选回答}

DPO 的原理：

- 不显式训练 Reward Model
- 而是通过改写 RL 目标，得到一个关于 LLM 的直接损失

数学直觉：

- 最大化： $\log P(y_{\text{win}}|x) - \log P(y_{\text{loss}}|x)$
- 这样就直接学会了"优选 vs 非优选"的差异

好处：

- 更简单（无需 Reward Model）
- 更快（少一个训练阶段）
- 同样有效（甚至更稳定，避免 reward hacking）

三种方案的对比

方案	数据	复杂度	效果	你的选择
SFT	{指令, 答案}	简单	中	✅ 现阶段用这个
RLHF	{指令, 答案} + 偏好标注	复杂	高	⌚ 后续尝试
DPO	{指令, 好答案, 差答案}	中等	高	⌚ 可尝试

◆ 参数高效微调（Parameter-Efficient Fine-Tuning, PEFT）

核心问题：Full Fine-Tuning 有什么问题？

Full FT 是最直接的方式：更新模型所有参数。

模型参数量：7B 显存占用（Full FT）：

- 模型权重（FP32）：28GB
- 梯度（FP32）：28GB
- 优化器状态（Adam）：56GB 总计：>100GB

结果：A100 80GB 显存爆炸！❌

LoRA：低秩适配（Low-Rank Adaptation）

基本原理

核心思想很优雅：大模型的权重矩阵虽然很大，但内在的秩（rank）很低。与其更新整个矩阵，不如只更新一个低秩的"增量"。

原始权重矩阵： $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$

LoRA 的做法：

- 冻结 W ，添加： $\Delta W = BA$
- 其中 $B \in \mathbb{R}^{d_{\text{out}} \times r}$ ， $A \in \mathbb{R}^{r \times d_{\text{in}}}$
- $r \ll d_{\text{out}}, d_{\text{in}}$ （秩很小）

前向传播：

$$y = Wx + \Delta Wx = Wx + BAx = Wx + B(Ax)$$

参数数量对比：

- **Full FT**: $d_{\text{out}} \times d_{\text{in}} \approx 7\text{B}$
- **LoRA** ($r = 32$): $2 \times 7\text{B} \times \frac{32}{\text{avg}_{\text{dim}}} \approx 0.6\%$ 的参数

显存对比：

- **Full FT**: $\sim 100\text{GB}$
- **LoRA** ($r = 32$): $\sim 15\text{-}20\text{GB}$ （只存 BA 的梯度）

🔗 深层理解：秩缺陷和 LoRA 设计

什么是矩阵的秩

秩 (Rank) = 矩阵中线性独立的行（或列）的最大个数

例子 1（满秩）：

$$W = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

- 秩 = 2（两行线性独立）
- 能代表整个 $\mathbb{R}^2$

例子 2（秩缺陷）：

$$W = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$$

- 秩 = 1（第二行是第一行的2倍）
- 只能代表一条直线

直觉：

- **秩** = 矩阵能"独立表达"的方向数
- **满秩** = 能充分利用所有维度
- **秩缺陷** = 大部分维度是"冗余"的

大模型中的秩缺陷

Qwen-VL-7B 的一个权重矩阵（注意力层中的投影矩阵）：

$$W \in \mathbb{R}^{4096 \times 4096}$$

理论上的秩：最多 4096

实验观察（来自 LoRA 论文）：在 Transformer 的不同层中测量有效秩 结果：秩往往只有理论最大秩的 1-10% 例如：一个 4096×4096 矩阵的有效秩可能只有 100-400

为什么？预训练阶段，模型学到的知识高度结构化 虽然参数维度很高，但信息集中在低维子空间 类比：虽然图片有百万像素，但可以用几个主成分解释大部分方差

秩缺陷的含义

含义 1：信息冗余 矩阵大部分维度是冗余的（高度相关） 只用小子集就能代表主要信息

含义 2：迁移学习机会 预训练已学到的"通用知识"占据主要秩 微调只需要改变很小的维度（低秩调整）

含义 3：参数高效微调的理论基础 不需要更新整个 W 只更新低秩的增量 $\Delta W = BA$ 就能达到接近完整微调的性能

为什么 AB 相乘能设定秩为 r

定理：

如果 $B \in \mathbb{R}^{d_{\text{out}} \times r}$, $A \in \mathbb{R}^{r \times d_{\text{in}}}$ 那么 $\text{rank}(BA) \leq \min(\text{rank}(B), \text{rank}(A), r)$

通常情况下, $\text{rank}(BA) = r$ （如果 B 和 A 都是满秩的）

通常情况下, $\text{rank}(BA) = r$ （如果 B 和 A 都是满秩的）

证明（直觉）：

- BA 的列向量都是 B 的列向量的线性组合
- B 只有 r 列
- 所以 BA 最多 r 个线性独立的列

例子：

- $B \in \mathbb{R}^{4096 \times 32}$ (32列)
- $A \in \mathbb{R}^{32 \times 4096}$ (32行)
- $BA \in \mathbb{R}^{4096 \times 4096}$ ，但秩最多 32

直觉画图：

$$\mathbb{R}^{4096} \xrightarrow{B(\text{映射})} \mathbb{R}^{32} \xrightarrow{A(\text{映射})} \mathbb{R}^{4096}$$

- B ：从高维压缩到低维（秩 ≤ 32 ）
- A ：从低维展开回高维（但依然最多 32 个独立方向）

为什么分别优化 A 和 B ，而不是直接优化 ΔW ？

这个问题问得非常好。有多个原因：

原因 1: 参数数量

直接优化 ΔW ：

- 参数数 = $4096 \times 4096 \approx 16\text{M}$

分别优化 A 和 B ($r = 32$):

- 参数数 = $(4096 \times 32) + (32 \times 4096) \approx 262\text{K}$
- 比例： $262\text{K}/16\text{M} \approx 1.6\%$

节省 98% 的参数！

原因 2: 显存和速度

显存占用：

- 直接优化 ΔW 的梯度： $16\text{M} \times 4 \text{ bytes} = 64\text{MB}$
- 优化 A, B 的梯度： $262\text{K} \times 4 \text{ bytes} \approx 1\text{MB}$
- 下降 64 倍

计算速度：

- 梯度更新的时间与参数数量成正比
- 优化 A, B 快 ~ 100 倍

原因 3: 数值稳定性和优化易用性

直接优化满秩矩阵 $\Delta W \in \mathbb{R}^{4096 \times 4096}$ ：

- 参数众多，相互作用复杂
- 优化景观很复杂，容易陷入局部最优
- 梯度方向众多且可能相互冲突

通过秩约束优化 A, B ：

- 强制解在低秩子空间
- 优化景观更平滑
- 更容易收敛到好的解

这类似于"正则化"的思想

原因 4: 初始化的巧妙性

标准 LoRA 初始化：

- $A \sim \mathcal{N}(0, \sigma^2)$ (高斯初始化)
- $B = 0$ (零初始化)

意义：

- 初始时 $\Delta W = BA = 0$
- 模型行为完全不变（从预训练模型开始）
- 训练过程中逐步偏离

这是对预训练知识的"尊重"：

- 保留预训练的所有知识
- 只做最小化的调整

为什么不直接约束 ΔW 的秩？

直接约束秩的问题：

- 秩约束是非凸的（优化很困难）
- 没有有效的算法

通过 AB 分解：

- 转化为凸优化问题（或接近凸）
- A, B 都是连续变量，可用标准梯度下降
- 自动满足秩约束 ($\text{rank}(BA) \leq r$)

这是一个经典的参数化技巧

LoRA 的关键超参数

超参	含义	推荐值	说明
rank (r)	低秩矩阵的秩	32-64	秩越高，性能越好，但显存/速度开销也大
alpha (α)	缩放因子	16-64	通常 $\alpha=2r$ ，控制 LoRA 增量的大小
target_modules	哪些层用 LoRA	q_proj, v_proj	通常只在注意力层和 FFN 的某些部分用
lora_dropout	LoRA 的 dropout	0.05	防止过拟合

你的 LoRA 配置

rank=32, alpha=64

- 效果：显存从 ~100GB 降到 ~20GB
- 性能：接近 Full FT（实验表明 r=32 已经很好）
- 成本：可以放在 A100×8 上训练

QLoRA：进一步压缩

QLoRA = Quantization + LoRA

核心思想：

1. 模型权重用 Int4 量化存储
2. 计算时反量化回 FP32/BF16
3. LoRA 增量用完整精度

效果：

- 4-bit 量化：显存进一步下降 4 倍
- 7B 模型可以放在单张 24GB GPU 上

你的情况：A100 80GB，Full FT 完全够 QLoRA 主要用于更极端的约束（如 3090、T4 等）暂时不必要

参数高效方案决策树

显存充足吗？

```
├ YES (>60GB for 7B)
│   └ 用 Full FT 或 LoRA (都可以)
├ NO (<30GB)
│   └ 用 LoRA (rank=32)
│       └ 还不够？用 QLoRA
```

第三阶段：推理与解码策略 (Inference & Decoding)

核心问题：模型训练好了，怎么让它开口说话？

训练好的 LLM 本质上是一个**概率预测机器**：给定前面的 tokens，预测下一个 token 的概率分布。但如何从这个概率分布中选择 token，决定了模型最终的输出质量。这就涉及到了**解码策略 (Decoding Strategy)**。

1. 基础解码：Greedy Search (贪婪搜索)

最简单的策略：**每次都选概率最大的那个 token。**

- **优点**：简单、确定性（每次输出都一样）。

- **缺点：**容易陷入循环重复；缺乏多样性；生成的文本往往过于平淡，甚至在长文本中出现退化。
- **适用场景：**数学题、代码生成等要求唯一标准答案的任务。

2. 进阶解码：Sampling (采样)

为了让文本更生动、更多样，我们不只是选概率最大的，而是根据概率分布进行**随机采样**。但这引入了一个问题：如果不加控制，可能会采样到概率很低、完全不通顺的词。

于是引入了两个关键参数来控制随机性：

🔥 Temperature (温度)

调整概率分布的形状。

- $T < 1$ (**低温**)：拉大概率差距，让高概率的词概率更高，模型更保守、更自信。
- $T > 1$ (**高温**)：缩小概率差距，让分布变平缓，低概率词也有机会被选中，增加创造性。

✂️ Top-p (Nucleus Sampling)

既然我们怕采样到尾部的垃圾词，不如直接把尾部砍掉。

- **机制：**累加概率最高的词，直到累加和达到 p (例如 0.9)。只在这部分核心词汇中采样。
- **效果：**既保留了多样性，又通过截断尾部保证了语言的连贯性。这是目前最主流的生成策略。

💡 核心机制：从数学角度看随机性 (Step-by-Step Mechanism)

从模型输出到最终选择，其实经历了 **4 个步骤**。假设词表只有 4 个词：[好, 不错, 糟糕, 这里的]。

Step 1: 原始输出 (Logits) 模型最后一层输出的原始数值（未归一化，范围 $-\infty \rightarrow +\infty$ ）：

$$z = [2.0, 1.0, 0.1, -5.0]$$

Step 2: 温度调整 (Temperature Scaling) 公式： $z' = z/T$

- 如果 $T = 1.0$ (默认): $z' = [2.0, 1.0, 0.1, -5.0]$
- 如果 $T = 0.5$ (降温): $z' = [4.0, 2.0, 0.2, -10.0]$ (差距被放大了!)
- 如果 $T = 2.0$ (升温): $z' = [1.0, 0.5, 0.05, -2.5]$ (差距被缩小了!)

Step 3: 归一化 (Softmax) 公式: $P_i = \frac{e^{z'_i}}{\sum e^{z'_j}}$

- $T = 1.0$ 时: $P \approx [0.71, 0.26, 0.03, 0.00]$
- $T = 0.5$ 时: $P \approx [0.88, 0.11, 0.01, 0.00]$ (强者更强)
- $T = 2.0$ 时: $P \approx [0.53, 0.32, 0.14, 0.01]$ (趋于平坦)

Step 4: 截断 (Top-p / Nucleus Sampling) 假设我们用 $T = 1.0$ 得到 $P = [0.71, 0.26, 0.03, 0.00]$, 设置 $p = 0.9$ 。

1. **排序:** 已经按概率降序排列。

2. **累加:**

- 词1("好"): $0.71 < 0.9$ (保留)
- 词2("不错"): $0.71 + 0.26 = 0.97 > 0.9$ (保留, 并不立即停止, 而是包含这一个让累计概率刚刚超过阈值的词)
- 词3("糟糕"): 剔除

3. **重归一化:** 只剩下 [好, 不错], 概率和为 0.97。将它们重新缩放为 100%。

$$P'_{\text{final}} = \left[\frac{0.71}{0.97}, \frac{0.26}{0.97} \right] \approx [0.73, 0.27]$$

4. **随机采样:** 从重归一化后的分布中随机抽取一个 token。

生成一个 0 到 1 之间的随机数, 如果落在 $[0, 0.73]$ 区间则选“好”, 落在 $[0.73, 1.0]$ 则选“不错”。

直观理解: 一场掷骰子的游戏 (Visual Analogy)

如果你觉得上面的数学太枯燥, 看看这个例子:

想象模型预测: "今天天气真__"。我们手中的工具是一个 **100 面的骰子**。

1. 原始概率 (默认状态)

- **好 (60%)**: 占据骰子的 1-60 面
- **不错 (20%)**: 占据 61-80 面
- **糟糕 (10%)**: 占据 81-90 面
- **其他垃圾词**: 91-100 面

 **直接掷骰子**: 如果不加控制, 有 10% 几率投出"糟糕", 甚至可能投出垃圾词。

2. Temperature 的角色 (分蛋糕的人) 在掷骰子前, Temperature 先把概率 (蛋糕) 重新切一遍:

- **T=0.1 (偏心)**: 强行把"好"的概率放大到 99%。骰子几乎只能投到它 (变得极其稳定, 行为接近 Greedy Search) 。
- **T=2.0 (疯癫)**: 强行拉平, "糟糕"的概率可能变大到 30% (变得胡言乱语) 。

3. Top-p 的角色 (安检员) Temperature 切完后, Top-p 站在门口安检:

- **规则**: 只让累计概率达标的前几名进场。
- **设 $p=0.8$** : "好"(60%) + "不错"(20%) 进场, 累计刚好 80%。
- **结果**: 大门关闭。"糟糕"和"垃圾词"直接被扔掉, 根本没机会参与掷骰子。

✅ **结论** 每次回答不一样, 是因为最后都在**掷骰子**。但在掷骰子之前, **Temperature 决定了骰子各面有多大, Top-p 决定了保留哪几面**。只要这两个参数不变, 候选的“骰子面”及其概率分布就是固定的, 随机性仅来自于最后的采样动作。

3. 拓展: 更高级的策略 (Extensions)

我们在 Week 7 会详细讨论更复杂的解码方法, 这里简要提及:

- **Beam Search (束搜索)**: 不只看这一步, 而是同时保留前 K 个最好的路径, 看哪条路径**整体**概率最高。常用于翻译任务, 但在开放式对话中容易生成空洞的重复内容。
- **Contrastive Search (对比搜索)**: 在生成时引入惩罚项, 强制模型不去重复之前的内容, 显著提升多样性。



Tip

做个预告：实际上，现在的推理不仅仅是简单的 Sampling。为了追求极智的速度，业界引入了 **Speculative Decoding (推测解码)** —— 用一个小模型先"猜"几个词，大模型再来验证。这将在后面的"推理优化"章节详细展开。

第四阶段：推理优化与部署

核心问题：微调后的模型怎么快速部署？

微调完成后，你需要在生产环境用这个模型。但直接用可能很慢。

瓶颈分析

自回归生成：LLM 一次生成一个 token

例子：生成 100 个 token 的回答 Step 1: 输入 prompt, 输出 token 1 计算量：moderate, 显存占用：high Step 2: 输入 [prompt + token 1], 输出 token 2 计算量：moderate, 显存占用：high ... Step 100: 输入 [prompt + ... + token 99], 输出 token 100

问题：

- 每一步都要重新计算 attention
- 显存占用很高（特别是长序列）
- 吞吐量低（每秒只能生成几个 token）

◆ 优化方案 1：KV Cache

标准 Attention 的计算过程

输入：Query Q, Key K, Value V

- $Q \in \mathbb{R}^{\text{batch} \times \text{seq}_{len} \times d_k}$
- $K \in \mathbb{R}^{\text{batch} \times \text{seq}_{len} \times d_k}$
- $V \in \mathbb{R}^{\text{batch} \times \text{seq}_{len} \times d_v}$

计算过程：

1. Step 1: 计算相似度

$$\text{scores} = QK^T \in \mathbb{R}^{\text{diff}_{len} \times \text{seq}_{len}}$$

2. Step 2: 归一化

$$\text{attention}_{weights} = \text{softmax}(\text{scores} / \sqrt{d_k}) \in \mathbb{R}^{\text{seq}_{len} \times \text{seq}_{len}}$$

3. Step 3: 加权求和（求和很关键）

$$\text{output} = \text{attention}_{weights} \cdot V \in \mathbb{R}^{\text{seq}_{len} \times d_v}$$

输出： $\text{output} \in \mathbb{R}^{\text{batch} \times \text{seq}_{len} \times d_v}$

计算量： $O(\text{seq}_{len}^2)$ 用于 QK^T 和注意力权重的矩阵乘法 显存占用： $O(\text{seq}_{len}^2)$ 用于存储 scores 矩阵 $\leftarrow$ 长序列时爆炸

自回归生成中的问题

自回归生成：一次生成一个 token

第 t 步：

- 已有输入序列： $[token_1, token_2, \dots, token_{t-1}]$
- 生成： $token_t$
- Q_t = 仅由 $token_t$ 提供
- $K_{1:t-1}$ = 由所有历史 token 提供
- $V_{1:t-1}$ = 由所有历史 token 提供

需要计算：

$$\text{scores} = Q_t K_{1:t-1}^T \in \mathbb{R}^{t-1}$$

$$\text{output}_t = \text{softmax}(\text{scores}) \cdot V_{1:t-1}$$

第 $t + 1$ 步：

- 已有序列： $[token_1, \dots, token_{t-1}, token_t]$
- 生成： $token_{t+1}$

需要计算：

$$\text{scores} = Q_{t+1} K_{1:t}^T \in \mathbb{R}^t$$

$$\text{output}_{t+1} = \text{softmax}(\text{scores}) \cdot V_{1:t}$$

⚠ 注意： $K_{1:t-1}$ 和 $V_{1:t-1}$ 已经在上一步算过了！为什么要重新算？

🔗 深层理解：KV Cache 的解决方案

关键观察

在第 t 步已经算过的 $K_{1:t-1}$ 和 $V_{1:t-1}$ 在第 $t+1$ 步不会改变

这正是"因果性"的体现：新 token 的注意力机制不会改变过去的计算结果 只需要把过去的结果"缓存"起来

你可能会有的疑惑

一个自然的问题："既然我只需要第 i 个 token 的预测，那我只拿 Q_i 去点乘所有的 $K_{0..i}$ 得到长度为 i 的注意力向量，然后加权求和 $V_{0..i}$ 不就行了？为什么要重新算一遍？"

这个问题触及了一个深层的认识：**模型架构与实现的"死板"特性**。答案需要从两个角度理解：

为什么"不加 KV Cache"时必须重算所有层

第一层问题：特征的多层耦合

Q_i 并非直接由原始 token 生成，而是经过了 80 层（对于大模型）的复杂计算链：

原始输入 X_i

↓ (Layer 1)

隐藏表示 $H_i^{(1)}$ （此时已融合了其他 token 的信息）

↓ (Layer 2)

隐藏表示 $H_i^{(2)}$

↓ (中间各层) ...

↓ (Layer 80)

最终隐藏表示 $H_i^{(80)}$

↓ (投影 $W_Q\beta$)

$Q_i \leftarrow$ 最终结果

关键点： 在第 40 层，第 i 个 token 的特征已经通过注意力融合了前面 token 的信息。你无法在"不输入前面 token"的情况下直接得到 Q_i ——因为 Q_i 本身就依赖于对整个序列的理解。

第二层问题：GPU 的"死板"实现逻辑

虽然从逻辑上我们"只需要最后一行"，但 GPU 和深度学习框架是这样设计的：

标准 Transformer 实现（训练时优化）
输入：序列 [token_1, ..., token_n]
内部执行：对整个序列进行矩阵化操作
• $Q \in \mathbb{R}^{(n, d)}$ （所有 n 个 token）
• $K \in \mathbb{R}^{(n, d)}$
• $V \in \mathbb{R}^{(n, d)}$
• $\text{Attention} = Q @ K^T$ （ $n \times n$ 矩阵）
输出：序列 [output_1, ..., output_n]
我们只取最后一个：output_n

为什么 GPU 要"全量计算"？

- GPU 的优势在于大规模并行矩阵乘法（GEMM）
- 从 GPU 的角度，计算 1 个 token 的 Q 和计算 1000 个 token 的 Q 的代价差不多（因为都要启动同一个 kernel，数据对齐）
- 框架的默认逻辑是**"全量并行"**，这在训练时是最高效的

所以，当你告诉模型"我要预测第 1000 个 token"时，框架的默认行为是：

1. 把 1000 个 token 全部喂进去
2. 在 80 层里面全部走一遍
3. 计算出 1000 个 output（虽然你只需要最后一个）

这完全是一种计算资源的浪费——但这正是 KV Cache 要解决的问题。

代码层面的对比

不加 KV Cache（伪代码）：

```
def generate_without_kv_cache():
    input_ids = [token_1] # 初始 prompt

    for step in range(n):

        # 每次都把累积的所有 token 重新喂给模型！

        outputs = model(input_ids) # 内部执行完整的前向传播

        # 只取最后一个位置的预测

        next_token_logits = outputs[:, -1, :]
        next_token_id = argmax(next_token_logits)

        # 累积 token

        input_ids.append(next_token_id)

    # 复杂度分析：

    # Step 1: 模型处理 1 个 token    →  $O(1)^3 = O(1)$ 

    # Step 2: 模型处理 2 个 token    →  $O(2)^3 = O(8)$ 

    # Step 3: 模型处理 3 个 token    →  $O(3)^3 = O(27)$ 

    # ...

    # Step n: 模型处理 n 个 token    →  $O(n)^3$ 
```

```
# 总计:  $O(n^3)$  ← 三层循环 (特征层、前向层、注意力点积)

# 现象: 生成 1000 个 token 时, 最后一步要处理 1000 个 token

# 这意味着第 1 个 token 的 Attention、MLP 等被重复计算了 1000 次!
```

加了 KV Cache (伪代码) :

```
def generate_with_kv_cache():
    input_ids = [token_1]
    kv_cache = None # 新增: 缓存过去的 K、V

    for step in range(n):

        # 每次只把新增的 1 个 token 喂给模型

        outputs, new_kv_cache = model(
            input_ids[-1:], # 只有最新的那个 token!
            past_kv_cache=kv_cache # 复用过去的 K、V
        )

        next_token_logits = outputs[:, -1, :]
        next_token_id = argmax(next_token_logits)

        input_ids.append(next_token_id)
        kv_cache = new_kv_cache # 更新缓存

# 复杂度分析:

# Step 1: 模型处理 1 个 token →  $O(1)$ 

# Step 2: 模型处理 1 个 token →  $O(2)$  (只算 Q 与 2 个 K 的点积)

# Step 3: 模型处理 1 个 token →  $O(3)$ 

# ...

# Step n: 模型处理 1 个 token →  $O(n)$ 
```

总计: $O(n^2)$ ← 两层循环 (生成步数、注意力点积)

现象: 每一步都只处理 1 个 token

第 1 个 token 的 Attention、MLP 只被计算一次, 然后"记住"了结果

KV Cache 的物理意义

【不加 KV Cache】

Step 1	处理 1 个 token
$Q_1 @ K_1$	→ Attention: 1×1 矩阵, MLP 也要算 1 次

Step 2	处理 2 个 token (含重复!)
$[Q_1, Q_2]$	→ Attention: 2×2 矩阵, MLP 也要算 2 次 (包括冗余的 token_1)
$@ [K_1, K_2]$	

Step 3	处理 3 个 token
$[Q_1, Q_2,$	→ Attention: 3×3 矩阵, MLP 也要算 3 次
$Q_3] @$	
$[K_1, K_2,$	
$K_3]$	

问题: token_1 和 token_2 被重复计算了很多次!

【加了 KV Cache】

Step 1	处理 1 个 token
$Q_1 @ K_1$	→ 算一次, 结果存进 cache
缓存 (K_1, V_1)	

Step 2	处理 1 个新 token (从 cache 读历史)
$Q_2 @$	
$[K_1(\text{cache}),$	→ 只算 token ₂ 的 feature, 与缓存的历史点积
$K_2(\text{new})]$	
缓存更新到	
(K_1, K_2, V_1, V_2)	

Step 3	处理 1 个新 token
$Q_3 @$	
$[K_1(\text{cache}),$	→ 只算 token ₃ 的 feature, 与缓存的历史点积
$K_2(\text{cache}),$	
$K_3(\text{new})]$	
缓存更新	

优势: 每个 token 的计算只发生一次, 然后被"记住"

KV Cache 的代价

显存占用:

必须存储所有历史的 K、V

对于 Qwen-VL-7B (32 层, 每层 32 个头, head_dim=128) :

每一层的 KV: $2 \times \text{seq_len} \times 4096 \times 2 \text{ bytes (BF16)} \approx 32 \text{ MB}$

32 层总计: $\approx 1 \text{ GB for seq_len}=1000$

权衡:

利: 避免重复计算, 加速 5-10 倍

弊: 显存占用明显 (但通常可接受)

KV Cache 的做法

在生成过程中, 维护一个 KV Cache:

```
kv_cache = {
    'layer_1': {'k': K_{1:t}, 'v': V_{1:t}},
    'layer_2': {'k': K_{1:t}, 'v': V_{1:t}},
```

```
...
'layer_32': {'k': K_{1:t}, 'v': V_{1:t}},
}
```

流程：

初始化: `kv_cache = None`

```
for t in range(1, seq_len):
    for layer in model.layers:
        Q_t = layer.W_q @ input_t  ← 只计算当前 token 的 Q
        K_t = layer.W_k @ input_t  ← 只计算当前 token 的 K
        V_t = layer.W_v @ input_t  ← 只计算当前 token 的 V

        if kv_cache is None:

            # 第一步，没有历史

            k_all = K_t
            v_all = V_t
        else:

            # 后续步骤，拼接缓存和新增

            k_all = concat(kv_cache['k'], K_t)  ← 关键：只算 K_t，其他读缓存
            v_all = concat(kv_cache['v'], V_t)  ← 关键：只算 V_t，其他读缓存

        # Attention (现在 Query 只有 1 个，而非 t 个)

        scores = Q_t @ k_all^T  ← Q_t 是向量 (1,d)，k_all 是矩阵 (t,d)
        output_t = softmax(scores) @ v_all

        # 更新缓存 (存储本层的所有 K、V 以备下一步使用)

        kv_cache['k'] = k_all
        kv_cache['v'] = v_all

    output_tokens.append(output_t)
```

为什么只缓存 K、V，不缓存 Q

这是一个很好的观察点！关键是 **Q** 的计算方式：

为什么 Q 必须重新计算：

$$Q_t = W_q \cdot \text{input}_t$$

input_t 只是当前生成的 token Q_t 只依赖于 input_t ，不依赖于历史 每一步的 Q 都不同（因为每一步的 input 都不同）→ 必须重新计算

为什么 K、V 需要缓存：

$$K_t = W_k \cdot \text{input}_t$$

$$V_t = W_v \cdot \text{input}_t$$

虽然计算公式相同，但关键不同之处是："在 Attention 中，我们需要用到历史"

在 Attention 中，我们计算：

$$\begin{aligned} \text{scores} &= Q_t \cdot [K_1, K_2, \dots, K_t]^T && \leftarrow \text{需要所有历史的 } K \\ \text{output} &= \text{softmax}(\text{scores}) \cdot [V_1, V_2, \dots, V_t]^T && \leftarrow \text{需要所有历史的 } V \end{aligned}$$

$$\begin{aligned} & \text{在上一步已算过，且永远不会变} \\ [V_1, \dots, V_{t-1}] & \text{ 在上一步已算过，且永远不会变} \end{aligned}$$

缓存它们，避免重新计算 → 这就是 KV Cache 的本质

时间复杂度分析（修正版）

💡 **核心洞察**：这里是最容易误导的地方。KV Cache **并不能** 把总复杂度从 $O(n^3)$ 降到 $O(n^2)$ ，而是通过改变 **Query** 的数量来优化。



不用 KV Cache 的情况

Step 1: 生成 token_1

- 输入：1 个 token
- 重新计算： K_1, V_1

- Query 产生: Q_1 (1个)
- 注意力计算: $Q_1 K_1^T$
 - 1个Query $\times$ 1个Key = 1 次点积
- 计算量: $O(1)$

Step 2: 生成 token_2

- 输入: 2 个 token (重新输入!)
- 重新计算: K_1, K_2, V_1, V_2 (全部重新算!)
- Query 产生: Q_1, Q_2 (2个) $\leftarrow$ 关键: 重新产生了 Q_1 !
- 注意力计算: $[Q_1, Q_2][K_1, K_2]^T$
 - 矩阵形状: $(2, d) \times (d, 2) = 2 \times 2$ 矩阵
 - 包括: $Q_1 K_1, Q_1 K_2, Q_2 K_1, Q_2 K_2$
- 计算量: $O(2^2) = O(4)$
 - 其中 $Q_1 K_1$ 在 Step 1 已经算过了! 但现在重新算了一遍

Step 3: 生成 token_3

- 输入: 3 个 token (重新输入!)
- 重新计算: $K_1, K_2, K_3, V_1, V_2, V_3$ (全部重新算!)
- Query 产生: Q_1, Q_2, Q_3 (3个) $\leftarrow$ 重新产生了 Q_1, Q_2 !
- 注意力计算: $[Q_1, Q_2, Q_3][K_1, K_2, K_3]^T$
 - 矩阵形状: $(3, d) \times (d, 3) = 3 \times 3$ 矩阵
 - 包括: 9 个点积 (其中 4 个在 Step 2 已算过)
- 计算量: $O(3^2) = O(9)$
 - 重复计算的比例越来越高

总计算量

注意力矩阵的总点积数:

$$1^2 + 2^2 + 3^2 + \dots + n^2 = \sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6} \approx \frac{n^3}{3}$$

K/V 的总计算次数:

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2} \approx \frac{n^2}{2}$$

总 FLOPs $\approx n^3/3 + n^2/2 \approx O(n^3)$ ($n^3/3$ 为主导项)

⇒ 时间复杂度: $O(n^3)$

例如 $n = 1000$:

- 注意力: $1000^3/3 \approx 3.33 \times 10^8$
 - KV: $1000^2/2 \approx 5 \times 10^5$
 - 总计: $\approx 3.33 \times 10^8$
-

有 KV Cache 的情况

Step 1: 生成 token_1

- 输入: 1 个 token
- 计算: K_1, V_1 (新增)
- 缓存: $[(K_1, V_1)]$
- Query 产生: Q_1 (1个)
- 注意力计算: $Q_1 K_1^T$
 - 1个Query $\times$ 1个Key = 1 次点积
- 计算量: $O(1)$

Step 2: 生成 token_2

- 输入: 1 个 token (只输入新的!)
- 计算: K_2, V_2 (仅新增, 不重新算 K_1, V_1 ✓)
- 缓存读: K_1, V_1
- 缓存更新: $[(K_1, V_1), (K_2, V_2)]$
- Query 产生: Q_2 (1个, 不重新产生 Q_1 !) ← 关键优化!
- 注意力计算: $Q_2 [K_1, K_2]^T$
 - 1个Query $\times$ 2个Key = 2 次点积
 - (没有计算 $Q_1 K_1, Q_1 K_2$, 因为那是 Step 1 的工作)
- 计算量: $O(2)$ ← 注意: 这不是 $O(2^2) = O(4)$

Step 3: 生成 token_3

- 输入：1 个 token（只输入新的！）
- 计算： K_3, V_3 (仅新增)
- 缓存读： K_1, K_2, V_1, V_2
- 缓存更新： $[(K_1, V_1), (K_2, V_2), (K_3, V_3)]$
- Query 产生： Q_3 (1个)
- 注意力计算： $Q_3[K_1, K_2, K_3]^T$
 - 1个Query $\times$ 3个Key = 3 次点积
 - （没有计算前两行，那些是 Step 1,2 的工作）
- 计算量： $O(3) \leftarrow$ 同样是 $O(3)$ ，不是 $O(3^2)$

总计算量

【Step i 的详细成本分解】

- 第 i 步产生的新 Query：1 个 (Q_i)
- 第 i 步使用的 Key 数量： i 个 ($K_1 \dots K_i$)
- 第 i 步使用的 Value 数量： i 个 ($V_1 \dots V_i$ ，来自缓存)
- 第 i 步的新注意力计算： $Q_i \times [K_1 \dots K_i]^T \cdot [V_1 \dots V_i]$
- 新 K/V 计算（每步）： $O(1)$
- 新注意力点积（每步）： $O(i)$
- 新注意力加权求和（每步）： $O(i) \leftarrow \text{weights}(1, i) \times V(i, d) = O(i \cdot d)$

但在大 O 记号中， $O(i)$ 和 $O(i \cdot d)$ 同级别（都是关于序列长度的线性项）

【总计算量】

注意力矩阵的总点积数 (QK^T):

$$1 + 2 + 3 + \dots + n = \sum_{i=1}^n i = \frac{n(n+1)}{2} \approx \frac{n^2}{2}$$

注意力加权求和数 (weights V):

$$1 + 2 + 3 + \dots + n = \sum_{i=1}^n i = \frac{n(n+1)}{2} \approx \frac{n^2}{2}$$

(这部分在两种情况下都是 $n^2/2$ ，因为都是向量与 Value 矩阵相乘)

K/V 的总计算次数：

$$1 + 1 + 1 + \dots + 1 = \sum_{i=1}^n 1 = n$$

总 FLOPs $\approx n^2/2(QK^T) + n^2/2(\text{weights } V) + n(\text{K/V 计算}) \approx n^2 + n \approx O(n^2)$

$\implies$ **时间复杂度：** $O(n^2)$

例如 $n = 1000$ ：

- QK^T 点积： $1000^2/2 \approx 5 \times 10^5$
- weights V 求和： $1000^2/2 \approx 5 \times 10^5$
- K/V 计算： $1000 \approx 1 \times 10^3$
- 总计： $\approx 1 \times 10^6$

💡 **关于 V 缓存的说明：** V 被缓存的主要意义是避免重新计算 V（特征提取），而不是避免点积。

在复杂度分析中：

- QK^T 这一步的 FLOPs 体现了 Query 数量的变化（无 i^2 ，有 i ）
- $\text{weights} \cdot V$ 这一步在两种情况下都是 $O(i)$ （1个权重向量乘以 $i \times d$ 矩阵）
- 但 V 本身的计算成本（特征提取）已经计入"K/V 计算"部分
 - 无 Cache： $\Sigma(1+2+\dots+n) = O(n^2)$
 - 有 Cache： $\Sigma(1+1+\dots+1) = O(n)$

对比总结

为什么两者复杂度会如此不同？

关键在于**Query** 数量的变化：

无 KV Cache: Step i 产生的 Query 数量 = i 个 ($Q_1, Q_2, \dots, Q_i$) 需要重新计算以前算过的行 每步计算的注意力点积 = i^2 个

有 KV Cache: Step i 产生的 Query 数量 = 1 个 (只有 Q_i) 以前的行已经固化, 直接读缓存 每步计算的注意力点积 = i 个

这正是从 $O(n^3)$ 降到 $O(n^2)$ 的物理原因。

完整成本对比

成本来源	无 KV Cache	有 KV Cache	优化效果
Step i 的 Query 数	i 个	1 个	$O(n^2)$ 倍的减少
每步注意力点积	i^2 个	i 个	$O(n)$ 倍的减少
K/V 重复计算	$1+2+\dots+n = O(n^2)$	$O(n)$	$O(n^2)$ 倍的减少
总注意力点积	$\sum i^2 = O(n^3/3)$	$\sum i = O(n^2/2)$	$O(n)$ 倍的减少
总 FLOPs	$O(n^3)$	$O(n^2)$	$O(n)$ 倍的减少

数值对比 (n=1000)

无 KV Cache: 注意力点积: $1000^3/3 \approx 3.33 \times 10^8$ K/V 重复: $1000^2/2 \approx 5 \times 10^5$ 总计: $\approx 3.33 \times 10^8$ FLOPs

有 KV Cache: 注意力点积: $1000^2/2 \approx 5 \times 10^5$ K/V 计算: $1000 \approx 1 \times 10^3$ 总计: $\approx 5 \times 10^5$ FLOPs

加速倍数 = $3.33 \times 10^8 / 5 \times 10^5 \approx 666$ 倍!

🔑 真实的 KV Cache 优化机制

从"矩阵-矩阵"变为"向量-矩阵"

无 KV Cache:

- 注意力 = $[Q_1, Q_2, \dots, Q_n][K_1, K_2, \dots, K_n]^T$
 - = $(n, d) \times (d, n)$ 的矩阵乘法
 - = 计算 n^2 个点积

有 KV Cache:

- 注意力 = $[Q_i][K_1, K_2, \dots, K_i]^T$ (每步)
 - = $(1, d) \times (d, i)$ 的向量乘矩阵
 - = 计算 i 个点积

累加:

$$1 + 2 + 3 + \dots + n = \frac{n(n + 1)}{2} \approx \frac{n^2}{2}$$

这就是复杂度从 $O(n^3) \rightarrow O(n^2)$ 的确切来源。

✅ 最终结论

	不用 KV Cache	有 KV Cache	加速
Query 产生方式	每步重新产生 1..i 个	每步只产生 1 个	$O(n^2)$
注意力计算	矩阵乘矩阵: $O(n^3/3)$	向量乘矩阵: $O(n^2/2)$	$O(n)$
K/V 重复计算	$O(n^2/2)$	$O(n)$	$O(n)$
总 FLOPs	$O(n^3)$	$O(n^2)$	$O(n)$
总时间复杂度	$O(n^3)$	$O(n^2)$	$O(n)$

你最初的理解与现在的差异

你原来说: "注意力计算应该从 $O(n^3)$ 降到 $O(n^2)$ "

- ✅ 现在你理解了: 这正是 KV Cache 的作用
- ✅ 物理原因: 通过限制 Query 数量为 1, 每步只计算当前 token 的注意力行向量
- ✅ 累加结果: $O(n^2)$ 而非 $O(n^3)$

关键洞察链：

1. 无 Cache：每步重新产生所有 Query $\rightarrow$ 矩阵乘矩阵 $\rightarrow O(n^2)$ per step $\rightarrow O(n^3)$ total
2. 有 Cache：每步只产生 1 个 Query $\rightarrow$ 向量乘矩阵 $\rightarrow O(i)$ per step $\rightarrow O(n^2)$ total
3. 节省的不只是"重复"，而是**改变了计算的维度**

显存占用

缓存一个序列的 K、V：

$$K \in \mathbb{R}^{(\text{seq_len}, \text{head_dim})} \quad V \in \mathbb{R}^{(\text{seq_len}, \text{head_dim})}$$

对于 7B Transformer 模型：通常 32 个头， $\text{head_dim} = 128$ 对应每一层都要缓存

总显存占用 (KV Cache)： $\sim \text{seq_len} \times \text{model_dim} \times 2 \times \text{bytes_per_value} \sim 1000 \times 4096 \times 2 \times 2$ (BF16) $\sim 16 \text{ MB per layer} \times 32 \text{ layers} \sim 0.5 \text{ GB for seq_len}=1000$

对于长序列 ($\text{seq_len}=4096$)： $\sim 2 \text{ GB}$

这就是为什么长序列推理时显存压力大

效果

不用 KV Cache：推理速度：50 tokens/sec 显存占用：高 延迟：中

用 KV Cache：推理速度：200-300 tokens/sec (4-6 $\times$) 显存占用：仍然高（但通常可接受） 延迟：低

◆ 优化方案 2：量化 (Quantization)

前置：浮点数表示

一个浮点数用三部分表示：符号位 (1 bit) | 指数位 (e bits) | 尾数位 (m bits) | = 总共 $1+e+m$ bits

📌 深层理解：不同精度格式对比

浮点数格式详解

FP32（单精度）：位数：1 | 8 | 23 = 32 bits 范围： $\sim 10^{-38}$ 到 10^{38} 精度：约 7 位十进制 用途：标准选择（训练、推理） 特点：范围大，精度高

FP16（半精度）：位数：1 | 5 | 10 = 16 bits 范围： $\sim 10^{-5}$ 到 10^4 （非常有限！） 精度：约 3 位十进制 用途：不推荐用于训练大模型 问题：范围太小，大模型容易溢出/下溢

BF16（Brain Float 16, Google 发明）：位数：1 | 8 | 7 = 16 bits 范围： $\sim 10^{-38}$ 到 10^{38} （和 FP32 一样！） 精度：约 3 位十进制（略低于 FP16 尾数部分） 用途：大模型训练的标准选择 优势：范围大（不溢出）+ 显存减半

Int8（8 位整数）：位数：8 bits 范围：-128 ~ 127 需要量化参数：scale, zero-point 精度：极低（但通过量化设计可以保证在可接受范围） 用途：推理优化

Int4（4 位整数）：位数：4 bits 范围：-8 ~ 7 需要量化参数：scale, zero-point 精度：极低 用途：极限压缩推理

比较表：浮点数与整数格式

格式	位数	指数位	尾数位	范围	精度	用途	优势	劣势
FP32	32	8	23	10^{-38} ~ 10^38	高（7位）	训练、推理	精度高、范围大	显存多、速度慢
FP16	16	5	10	10^{-5} ~ 10^4	中（3位）	推理	显存减半	范围小，容易溢出/下溢
BF16	16	8	7	10^{-38} ~ 10^38	中偏低（3位）	训练、推理	范围大、显存减半	精度略低于 FP16
Int8	8	—	—	-128~127	极低	推理	显存减4倍、速度快	精度损失 1-3%
Int4	4	—	—	-8~7	极低	推理	显存减8倍	精度损失明显

为什么 BF16 是大模型的标准

让我用一个具体例子说明 FP16 为什么不行：

例子：Attention 中 QK^T 的计算

- $Q \in \mathbb{R}^{\text{seq}_{len} \times d_{\text{model}}}$
- $K \in \mathbb{R}^{\text{seq}_{len} \times d_{\text{model}}}$
- $\text{scores} = QK^T \in \mathbb{R}^{\text{seq}_{len} \times \text{seq}_{len}}$

假设某个 score 值是 1000（很大）：

- **FP32**：完全没问题
- **FP16**：指数最大 = $2^{(2^4-1)} \approx 10^4$ ，1000 还在范围内
- **BF16**：完全没问题

假设某个 score 值是 100,000 (更大) :

- **FP32**: 完全没问题 (范围到 10^{38})
- **FP16**: 溢出到 inf $x \leftarrow$ 这就是问题!
 - 指数只有 5 bits, 能表示最大值约 10^4 , 100,000 超出范围
- **BF16**: 完全没问题 (指数 8 bits, 范围和 FP32 一样)

大模型中梯度的跨度很大:

- 前向传播中的值可能从 10^{-6} 到 10^6
- FP16 的指数只有 5 bits, 经常溢出
- 导致数值不稳定, 训练失败
- **BF16** 因为有 8 位指数位 (和 FP32 一样), 完全不会溢出
- 同时因为尾数只有 7 bits (vs FP16 的 10 bits), 精度略低
- 但大模型对精度没那么敏感, 因为参数众多
- 所以 **BF16 是完美的平衡**

具体数值对比

存储一个权重矩阵 (Qwen-VL 中的一个 4096×4096 矩阵) :

FP32: $4096 \times 4096 \times 4 \text{ bytes} = 64 \text{ MB}$

FP16 / BF16: $4096 \times 4096 \times 2 \text{ bytes} = 32 \text{ MB}$ (节省 50%)

Int8: $4096 \times 4096 \times 1 \text{ byte} = 16 \text{ MB}$ (节省 87.5%)

- 量化参数 (scale, zero-point) $\sim 8 \text{ bytes} \approx 4 \text{ MB}$

Int4: $4096 \times 4096 \times 0.5 \text{ byte} = 8 \text{ MB}$ (节省 93.75%)

- 量化参数 $\approx 2 \text{ MB}$

7B 模型的显存占用

模型大小 7B: FP32: 28 GB (仅权重) BF16: 14 GB (仅权重) Int8: 7 GB (推理)
Int4: 3.5 GB (推理)

训练时总显存 (加梯度+优化器状态): FP32: $28 + 28 + 56 = 112$ GB ← A100 不够!
BF16: $14 + 14 + 28 = 56$ GB ← A100 还有空间 BF16 on A100×8: 轻松, 还有很大空间
用于 batch size

推理时显存 (加激活值、中间结果): FP32: ~40-50 GB BF16: ~20-25 GB Int8: ~10-15 GB
Int4: ~7-10 GB (加 KV Cache 可能需要更多)

训练 vs 推理的量化策略

训练阶段 (精度优先):

权重: BF16 或 FP32

激活值: BF16

梯度: BF16

优化器状态 (Adam): FP32

→ 保持精度, 支持有效训练

→ 精度问题会导致 loss 不收敛或震荡

推理阶段 (速度优先):

权重: Int8 或 Int4 (量化)

激活值: FP16 或 BF16

无需梯度、无需优化器状态

→ 大幅加速和节省显存

→ 精度损失 1-3% 通常可接受

你的选择:

训练: BF16 (大模型标准)

推理: Int8 (平衡速度和精度)

Int4 vs Int8 量化的差异

Int8 量化 (标准方式):

线性量化公式:

$$x_{\text{int8}} = \text{round}(x_{\text{fp32}}/S) + Z$$

反量化：

$$x_{\text{fp32}} \approx (x_{\text{int8}} - Z) \cdot S$$

精度损失：

- 从 FP32 的 2^{32} 种值 $\rightarrow$ Int8 的 256 种值
- 量化误差 $\sim 1 - 2\%$
- 大模型可接受

NF4 (Normal Float 4, 在 QLoRA 中使用)：

- 针对神经网络权重分布优化
- 权重通常近似高斯分布
- NF4 用 4 bits 表示 (相比 Int4 更优的量化点)
- 相比均匀 Int4, 精度更好
- 相比 Int8, 显存节省更多

推荐方案：

- 训练：不量化 (用 BF16 保持精度)
- 推理：优先 Int8 (精度损失 1-2%)
- 推理 (极限压缩)：Int4 or NF4 (精度损失 3-5%)

优化方案 2 的效果

原始推理 (Baseline)：

- 模型大小：28GB (FP32)
- 推理速度：50 tokens/sec
- 显存占用：高
- 延迟：中

应用量化 (Int8)：

- 模型大小：7GB (1/4)
- 推理速度：75-100 tokens/sec ($1.5-2\times$)
- 显存占用：低
- 延迟：低

- 精度：F1 下降 1-3%

◆ 优化方案 3：批处理（Batching）

问题

单条请求生成太慢

解决

同时处理多条请求（batch）

效果：

- 吞吐量（throughput）从 10 tokens/sec 提升到 1000 tokens/sec
- 但延迟（latency）可能增加

权衡： 在线服务（如聊天）： 需要低延迟 → 小 batch 离线分析（如批量评估）： 需要高吞吐 → 大 batch

完整的推理优化栈

原始推理（Baseline）

- └ 模型大小：28GB（FP32）
- └ 推理速度：50 tokens/sec
- └ 显存占用：高
- └ 延迟：中

应用 KV Cache

- └ 推理速度：200 tokens/sec（4×）
- └ 显存占用：仍然高
- └ 延迟：低

应用量化（Int8）

- └ 模型大小：7GB（1/4）
- └ 推理速度：300 tokens/sec

- └ 显存占用：低
- └ 延迟：低

应用批处理 (batch=16)

- └ 吞吐量：3000 tokens/sec (单 GPU)
- └ 延迟：中-高 (取决于 batch size)
- └ 显存占用：中等

你的微调任务在全景中的位置

大模型完整生命周期

① 预训练

- Qwen-VL 官方已完成
- 你直接使用

② 微调与对齐 ← 你在这里！

你的任务：

- 模型：Qwen-VL-7B
- 数据：CameraBench VQA
- 方案：SFT (指令微调)
- 参数效率：LoRA (rank=32)
- 训练精度：BF16 (混合精度)
- 分布式：单机 8xA100
- 评估指标：VQA-specific (F1, BLEU)

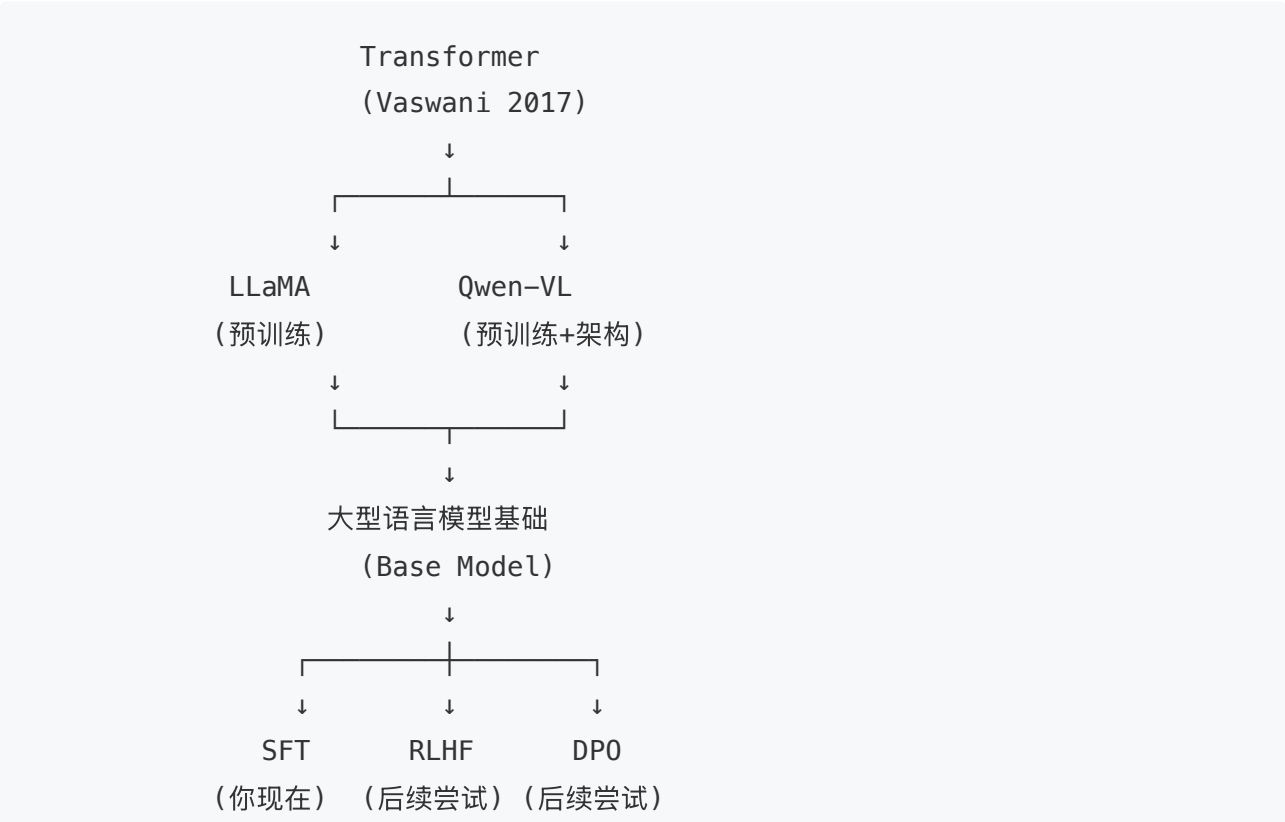
③ 推理优化 (后续)

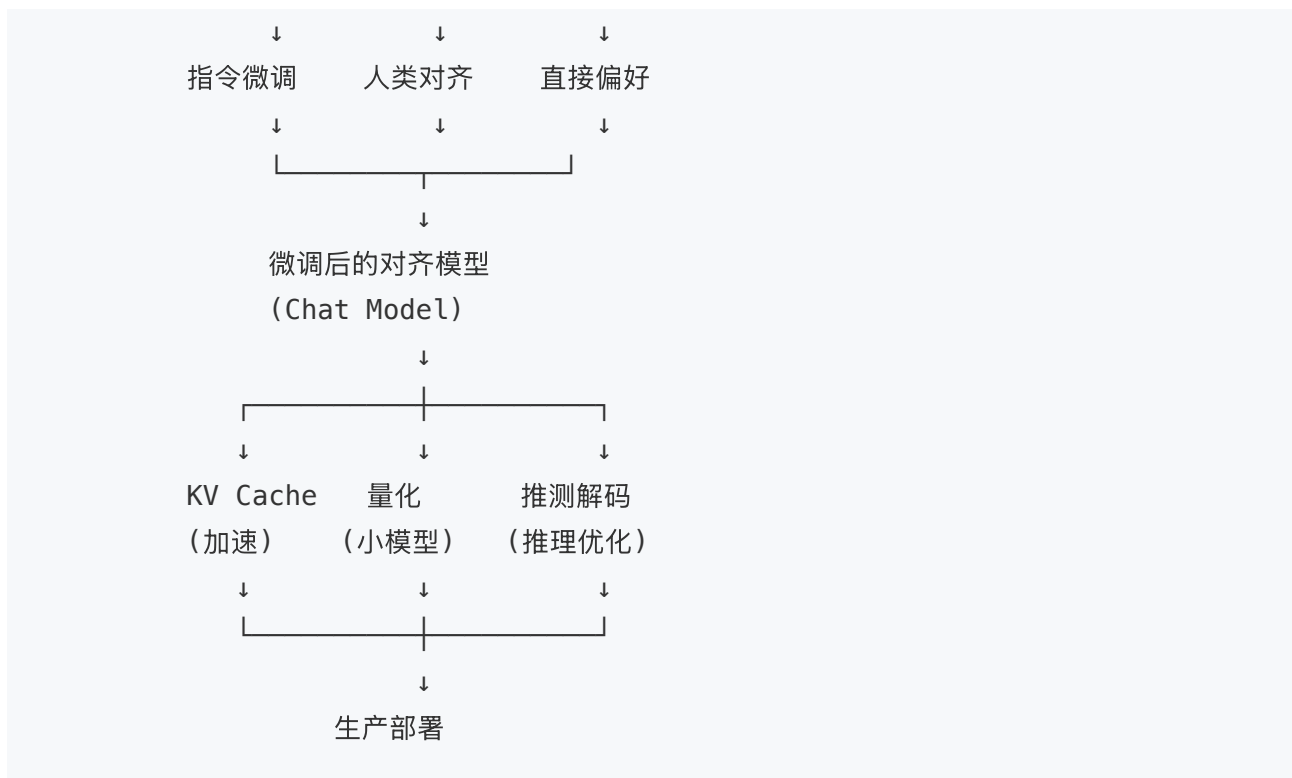
- 量化：Int8 推理
- KV Cache 优化
- 批处理部署

📄 核心决策表（为什么选这些）

决策	选项	理由
微调方案	SFT	CameraBench 有高质量标注，SFT 足够；RLHF/DPO 后续尝试
参数高效	LoRA (r=32)	A100 80GB 显存足够，LoRA 达到 ~99% 的 Full FT 性能
量化	无 (BF16即可)	训练时保持精度，推理时再考虑量化
精度	BF16	范围大（不溢出）、显存减半、大模型标准选择
分布式	单机 FSDP	A100×8 机器内部延迟低，FSDP 易于扩展

🔗 论文依赖关系图





你需要精读的论文：

1. **LoRA (Hu et al. 2021)** ← 决定你的微调策略
2. **Qwen-VL 技术报告** ← 理解你要微调的模型
3. **SFT 原理** (在 Llama 2 论文中讲得很清楚) ← 理解指令微调

后续可选：

4. **DPO (Rafailov et al. 2023)** ← 下一阶段的对齐方向
 5. **量化相关 (QLoRA)** ← 推理优化时需要
-

✔ Phase 0 小结

关键概念总结表

概念	你的理解	关键问题
预训练	Qwen-VL 用自监督学习学通用能力	为什么要微调而不直接用？
SFT	用高质量 QA 对教模型遵循指令	这就是指令微调
RLHF	通过人类反馈学习价值判断	为什么 SFT 不够？
LoRA	冻结原模型，只训练低秩增量	为什么 rank=32 就够？
BF16	16 位浮点数，范围大+精度平衡	为什么大模型都用这个？
KV Cache	缓存过去的 K、V，避免重复计算	这是推理优化的第一步
解码策略	控制如何从概率中选择 Token	为什么单纯取概率最大不行？
量化	用低位数表示权重，减小模型	推理时用 Int8，训练时用 FP32

你现在应该能回答

- ✔ 为什么选 SFT 而不是 RLHF？
- 答：CameraBench 有高质量标注，SFT 足够；RLHF 后续可尝试
- ✔ 为什么用 LoRA 而不是 Full FT？
- 答：A100 显存有限，LoRA 节省显存同时保持性能，秩缺陷是理论基础
- ✔ LoRA 的 rank 怎么选？
- 答：通常 rank=32-64，你选 32 作为 baseline，后续对比 64
- ✔ 训练时用什么精度？推理时呢？

- 答：训练用 BF16（范围大、显存减半），推理可用 Int8（进一步加速）
 - ☒ KV Cache 为什么有效？成本是什么？
 - 答：避免重复计算 K、V，加速 5-10 倍；但 KV Cache 本身占用显存
 - ☒ 为什么 AB 相乘能设定秩为 r ？
 - 答：秩界定理， $\text{rank}(BA) \leq r$ ；初始化 $B=0$ 使得初始增量为零
-

下一步：你要做什么

Week 1 准备

1. 理论侧完成 Phase 0 ☒（现在）

- 理解这份讲义，确保每个概念清晰
- 对不明白的地方提问

2. 工程侧准备（Week 1 开始）

- Clone Qwen 官方 repo
- 环境搭建
- 推理脚本验证
- 小数据微调验证

3. 预期产出

- 完整的环境配置
 - 官方推理脚本可跑
 - 理解 LoRA 超参数配置
 - 准备开始 SFT 微调实验
-

关键参考

-  学习计划：见《学习计划-正式版.md》
-  进度追踪：见《学习进度追踪.md》

-  文档导航：见《文档体系导航.md》